НАО «КАРАГАНДИНСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ АБЫЛКАСА САГИНОВА»

Кафедра Кибербезопасности и искусственного интеллекта

Лабораторная работа № 2

по дисциплине «Безопасность Баз данных»
Тема:«УПРАВЛЕНИЕ ДОСТУПОМ В СУБД SQL SERVER»
Специальность: 6В06301 – «Системы информационной безопасности»

Принял: ст.преп.
Жакина М.М.
Выполнил:
СИБ 23-7 Гарифуллин И.Р.

Караганда 2026

Задание №1. необходимо создать резервные копии базы данных «TestDB» с использованием полного резервного копирования, разностного резервного копирования и резервного копирования журнала транзакций.

Ход работы:

1. Запустите SQL Server Management Studio (SSMS), подключитесь к своему экземпляру SQL Server.
2. Создайте папку с именем c:\Student\test.
3. Откройте окно нового запроса. Измените контекст на базу данных master. Наберите и исполните следующую команду, чтобы создать полную резервную копию базы данных:

BACKUP DATABASE TestDB TO DISK = 'C:\TEST\AW.bak’

Ознакомьтесь с результатами запроса – какая информация обработана, сколько страниц, сколько файлов.

4. Внесите изменение в таблицу в вашей БД TestDB. Добавьте одну запись (придумайте сами)

5. Откройте окно нового запроса наберите и исполните следующую команду, чтобы создать резервную копию журнала транзакций и сохранить только что внесенное изменение:

BACKUP LOG TestDB TO DISK = ‘C:\TEST\AW1.trn'

Ознакомьтесь с результатами запроса – какая информация обработана, сколько страниц, сколько файлов.

6. Внесите еще одно изменение в таблицу.
7. Откройте окно нового запроса наберите и исполните следующую команду, чтобы создать разностную резервную копию базы данных:

BACKUP DATABASE TestDB TO DISK = ‘C:\TEST\AWDIFF1.bak' WITH DIFFERENTIAL

Ознакомьтесь с результатами запроса – какая информация обработана, сколько страниц, сколько файлов.

8. Внесите еще одно изменение в таблицу.
9. Откройте окно нового запроса наберите и исполните следующую команду, чтобы создать полную резервную копию базы данных в указанном месте на диске:

BACKUP LOG TestDB TO DISK = ‘C:\....TEST\AW2.TRN'

Ознакомьтесь с результатами запроса – какая информация обработана, сколько страниц, сколько файлов.

Задание 1. Создание резервных копий базы данных TestDB.

Сначала кратко о том, как это реализовано в Microsoft SQL Server. В этой системе резервное копирование встроено непосредственно в механизм сервера. Используются три основных типа резервных копий: полный backup (FULL),

разностный backup (DIFFERENTIAL) и резервная копия журнала транзакций (LOG). Полный backup копирует все страницы базы данных на момент выполнения команды. Разностный backup сохраняет только те страницы, которые изменились после последнего полного backup. Backup журнала транзакций сохраняет изменения из transaction log. Благодаря этому формируется цепочка восстановления: сначала восстанавливается полный backup, затем разностный backup, а затем последовательно применяются журналы транзакций. Команды выполняются через SQL или через интерфейс SQL Server Management Studio.

В PostgreSQL архитектура резервного копирования устроена иначе. Основная концепция основана на журнале WAL (Write Ahead Log). Все изменения базы данных сначала записываются в WAL, а затем применяются к файлам таблиц. Поэтому стандартная схема резервного копирования PostgreSQL состоит из двух частей: полного физического snapshot базы (base backup) и архивированных WAL-журналов. Base backup является копией всех файлов кластера базы данных, а WAL-архив содержит последовательность изменений, произошедших после создания backup. При восстановлении сначала используется base backup, затем сервер последовательно воспроизводит WAL-файлы и восстанавливает состояние базы.

Далее описаны этапы выполнения работы.

Сначала был запущен сервер PostgreSQL в контейнере.

docker run -d --name postgres-practice -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=admin -e POSTGRES_DB=practicedb -p 5433:5432 -v practicedata:/var/lib/postgresql/data postgres:16

После подключения была создана база данных TestDB.

CREATE DATABASE testdb;

Внутри базы была создана тестовая таблица.

```
CREATE TABLE students (
 id SERIAL PRIMARY KEY,
 name TEXT,
 grade INTEGER
 );

INSERT INTO students(name,grade) VALUES ('Ivan',5);
 INSERT INTO students(name,grade) VALUES ('Alex',4);
```

Следующим этапом была настройка архивирования WAL. Для этого необходимо изменить конфигурацию PostgreSQL. Сначала был выполнен вход в контейнер.

```
docker exec -it postgres-practice bash
```

В каталоге данных PostgreSQL была создана папка для хранения архивов WAL.

```
mkdir /var/lib/postgresql/data/wal_archive
```

Затем был открыт файл конфигурации сервера.

```
nano /var/lib/postgresql/data/postgresql.conf
```

В конфигурации были изменены следующие параметры.

```
wal_level = replica
 archive_mode = on
 archive_command = 'cp %p /var/lib/postgresql/data/wal_archive/%f'
```

Параметр wal_level включает полный журнал изменений базы. Параметр archive_mode включает режим архивирования WAL. Параметр archive_command задаёт команду, которая выполняется при завершении WAL-сегмента. В данном случае WAL-файл копируется в каталог wal_archive.

После изменения конфигурации контейнер был перезапущен.

```
docker restart postgres-practice
```

Далее в таблицу были добавлены новые записи, чтобы создать изменения в базе данных.

```
INSERT INTO students(name,grade) VALUES ('Maria',3);
 INSERT INTO students(name,grade) VALUES ('Olga',5);
```

Поскольку WAL-файлы архивируются только после завершения сегмента, была выполнена команда принудительного переключения WAL.

```
SELECT pg_switch_wal();
```

После этого в каталоге wal_archive появились архивные WAL-файлы. Проверка выполнялась внутри контейнера.

```
docker exec -it postgres-practice bash
```

```
ls /var/lib/postgresql/data/wal_archive
```

```
root@b62249f0f5b5:/# ls /var/lib/postgresql/data/wal_archive
000000010000000000000003
root@b62249f0f5b5:/# exit
```

Таким образом была получена последовательность WAL-файлов, содержащих изменения базы данных.

Следующим этапом был создан полный физический backup базы данных (base backup). Для этого используется утилита pg_basebackup. Команда выполнялась в терминале.

docker exec -t postgres-practice pg_basebackup -U admin -D /tmp/basebackup -Fp -Xs -P

Эта команда создаёт физическую копию всех файлов базы данных. Параметр -D указывает каталог для backup, -Fp означает формат обычных файлов, -Xs указывает, что необходимо включить WAL-файлы, а параметр -P показывает прогресс выполнения.

После создания backup папка была скопирована из контейнера на локальный компьютер.

docker cp postgres-practice:/tmp/basebackup ./basebackup

В результате была получена полная физическая копия базы данных и архив WAL-файлов. Такая схема резервного копирования является стандартной архитектурой PostgreSQL и используется для восстановления базы данных на любой момент времени.

Задание 2. Восстановление базы данных PostgreSQL из base backup + WAL.

Сначала кратко о том, как это работает в Microsoft SQL Server. В MSSQL восстановление базы выполняется через команду Restore database или через интерфейс SQL Server Management Studio. Пользователь выбирает файл резервной копии (full backup, differential или log backup) и указывает базу для восстановления. Сервер последовательно восстанавливает данные из полного backup, затем применяет разностные backup и журналы транзакций, что позволяет восстановить базу на любой выбранный момент времени. Все это делается встроенными средствами сервера без необходимости вручную работать с файлами базы.

В PostgreSQL восстановление устроено иначе. Архитектура PostgreSQL строится вокруг физического base backup и WAL-журналов. Восстановление базы осуществляется через замену файлов кластера базы на base backup и последовательное применение WAL-файлов, которые содержат все изменения

после создания backup. Для этого PostgreSQL использует файк recovery.signal и параметр restore_command, который указывает, как доставать WAL-файлы из архива. Такой подход позволяет восстановить базу до состояния на конкретный момент времени, но требует прямого доступа к файловой системе сервера или контейнера.

Шаг 1. Сохраняем base backup и WAL на хосте:

```
docker cp postgres-practice:/tmp/basebackup ./basebackup
docker cp postgres-practice:/var/lib/postgresql/data/wal_archive ./wal_backup
```

Шаг 2. Создаем новый контейнер PostgreSQL для восстановления, чтобы не трогать старый:

```
docker run -d --name postgres-restore -e POSTGRES_USER=admin -e POSTGRES_PASSWORD=admin -p 5434:5432 -v restoredata:/var/lib/postgresql/data postgres:16
```

Шаг 3. Останавливаем новый контейнер перед копированием файлов:

```
docker stop postgres-restore
```

Шаг 4. Копируем base backup и WAL внутрь контейнера:

```
docker cp ./basebackup/. postgres-restore:/var/lib/postgresql/data/
docker cp ./wal_backup/. postgres-restore:/var /lib/postgresql/data/wal_archive
```

Шаг 5. Настраиваем восстановление внутри контейнера:

```
docker exec -it postgres-restore bash
vim /var/lib/postgresql/data/postgresql.conf
```

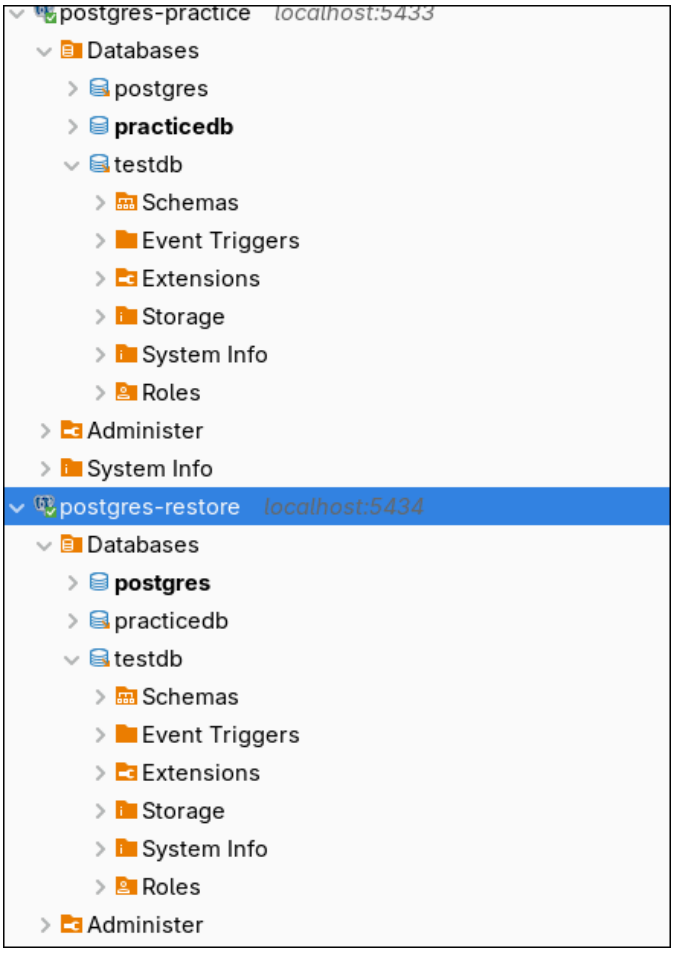
Шаг 6. в конфиг добавить строку

```
restore_command = 'cp /var/lib/postgresql/data/wal_archive/%f %p'
```

Шаг 7.

```
touch /var/lib/postgresql/data/recovery.signal
```

Как итог две одинаковые БД на разных портах



Задание №3. необходимо создать шифрованную резервные копии базы данных «TestDB» с использованием полного резервного копирования

Шифрование бэкапа

docker exec -t postgres-practice pg_dump -U admin testdb > testdb_full.sql

openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in testdb_full.sql -out testdb_full.sql.enc -k 123

Задание №4 и №5 – удаление мастер-ключа и сертификата. Восстановление зашифрованного БД

Расшифровка бэкапа и восстановление

openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in testdb_full.sql.enc -out testdb_full.sql -k 123

docker exec -it postgres-practice psql -U admin -c "DROP DATABASE IF EXISTS testdb;"

docker exec -it postgres-practice psql -U admin -c "CREATE DATABASE testdb;"

docker exec -i postgres-practice psql -U admin testdb < testdb_full.sql

Задание №6– разобраться в скриптах и написать пояснения каждого параметра в отчетах

Резервное копирование (Задание 1)

BACKUP DATABASE … TO DISK = 'путь'

Создаёт полную резервную копию всей базы данных на указанный момент времени.

WITH DIFFERENTIAL

Указывает, что создаётся разностная копия — сохраняются только изменения с последней полной копии.

BACKUP LOG … TO DISK = 'путь'

Создаёт резервную копию журнала транзакций — позволяет восстанавливать базу до конкретной точки во времени.

WITH INIT

Перезаписывает файл резервной копии (если он уже существует).

Восстановление базы данных (Задание 2)

RESTORE DATABASE … FROM DISK = 'путь'

Основная команда для восстановления базы из файла резервной копии.

WITH REPLACE

Разрешает перезаписывать существующую базу данных с таким же именем.

WITH RECOVERY

Завершает восстановление, приводит базу в рабочее состояние (ONLINE), откатывает незавершённые транзакции.

WITH NORECOVERY

Оставляет базу в состоянии Restoring — позволяет дальше применять разностные копии и логи транзакций.

WITH MOVE 'логическое_имя' TO 'новый_путь'

Указывает новое расположение или имя файла данных / лога при восстановлении (нужно, когда восстанавливаем в новую базу или пути заняты).

Шифрование резервных копий (Задания 3–5)

CREATE MASTER KEY ENCRYPTION BY PASSWORD = '...'

Создаёт главный ключ базы master, который защищает сертификаты и ключи шифрования.

CREATE CERTIFICATE … WITH SUBJECT = '...'

Создаёт сертификат, который будет использоваться для шифрования резервных копий.

BACKUP DATABASE … WITH ENCRYPTION (ALGORITHM = AES_256, SERVER CERTIFICATE = ...)

Шифрует сам файл резервной копии с помощью алгоритма AES-256 и указанного сертификата.

BACKUP CERTIFICATE … TO FILE = '...' WITH PRIVATE KEY (...)

Сохраняет сертификат и его приватный ключ в файлы для последующего переноса или восстановления.

CREATE CERTIFICATE … FROM FILE = '...' WITH PRIVATE KEY (FILE = '...', DECRYPTION BY PASSWORD = '...')

Восстанавливает сертификат из ранее сохранённых файлов (нужен тот же пароль, которым ключ был зашифрован).

Основные выводы по шифрованию

Без сертификата с тем же отпечатком (thumbprint) восстановить зашифрованную копию невозможно → ошибка 33111.

Это демонстрирует защиту: даже при наличии файла .bak данные остаются недоступными без правильных ключей.